

Blackthorn

Security Review For Morpho

Collaborative Audit Prepared For: **Morpho**

Lead Security Expert(s): **0x73696d616f**

hyh

xiaoming90

Date Audited: **July 14 - July 21, 2026**

Introduction

The bundles repository contains contracts that interact with Morpho protocols on behalf of the user. This is useful to do onchain checks such as slippage checks, batch calls that potentially depend on each other, and enable callbacks such as when doing a flashloan.

This audit focuses on Blue bundles, Vault bundles and Vault Exit bundles.

Blue bundles are meant to provide the bundles to interact with Morpho Blue, which includes the migration of a borrow position from one market to another. Vault bundles target the Morpho vaults, that is vault v1 (including v1.1) and vault v2. Vault Exit bundles allow the user to exit a Morpho vault, either through force withdrawals or through in-kind redemption. Force withdrawals return assets to the user while in-kind redemptions return positions on the underlying markets of the vault but guarantee that the user can exit the vault.

Scope

Repository: [morpho-org/bundles](https://github.com/morpho-org/bundles)

Audited Commit: [713af6ddfa64270c4ac8689532f05306a8ac72ea](https://github.com/morpho-org/bundles/commit/713af6ddfa64270c4ac8689532f05306a8ac72ea)

Final Commit: [9994e6abe5b18d5f7e0d6bd666f85eb259e3312f](https://github.com/morpho-org/bundles/commit/9994e6abe5b18d5f7e0d6bd666f85eb259e3312f)

Files:

- [src/blue/BlueBundlesV1.sol](#)
- [src/blue/interfaces/IBlueBundlesV1.sol](#)
- [src/libraries/interfaces/IERC20Permit.sol](#)
- [src/libraries/TokenLib.sol](#)
- [src/vault-force-withdraw/interfaces/IVaultForceWithdrawBundlesV1.sol](#)
- [src/vault-force-withdraw/VaultForceWithdrawBundlesV1.sol](#)
- [src/vault/interfaces/IVaultBundlesV1.sol](#)
- [src/vault/VaultBundlesV1.sol](#)

Final Commit Hash

9994e6abe5b18d5f7e0d6bd666f85eb259e3312f

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	7

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Security Experts Dedicated to This Review

@0x73696d6166

A handwritten signature in white ink on a black background. The signature consists of a large, stylized capital 'S' that loops around the word 'Simao', which is written in a cursive script.

@hyh

A handwritten signature in white ink on a black background. The signature is a stylized, cursive 'hyh'.

@xiaoming90

A handwritten signature in white ink on a black background. The signature is a stylized, cursive 'Xiaoming90'.

Issue L-1: Directly whitelisted VaultBundles V1 bypasses Vault V2 send-assets gate [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/issues/9>

Vulnerability Detail

The comment at Line 16 mentioned that if the vault is gated, it should permit/whitelist this VaultBundlesV1 contract.

```
File: VaultBundlesV1.sol
16: /// @dev Gated vaults (Vault V2) require this contract to be permitted by the
    ↪ relevant gates.
17: /// @dev This contract can approve tokens to arbitrary addresses. This is safe
    ↪ because a token amount pulled is always fully spent in the same transaction,
    ↪ and because the only tokens pulled to this contract are owned by msg.sender.
18: /// @dev No-ops are not systematically prevented.
19: /// @dev Zero checks are not systematically performed.
20: contract VaultBundlesV1 is IVaultBundlesV1 {
```

So, the Vault V2's owner has to whitelist the VaultBundlesV1 contract address AND set isIntermediary to False. isIntermediary cannot be set to True because the VaultBundlesV1 does not support or expose the initiator() function.

```
File: WhitelistSendAssetsGate.sol
46:     /// @dev Reverts if isIntermediary[account] but account reverts on
    ↪ initiator().
47:     function canSendAssets(address account) external view returns (bool) {
48:         return isWhitelisted[isIntermediary[account] ?
    ↪ IIntermediary(account).initiator() : account];
49:     }
```

For the receive-shares gate, the gate has the ability to determine the caller who calls the

VaultBundlesV1 because the `onBehalf` is equal to the `msg.sender` (caller), and `onBehalf` is passed into the gate.

```
File: VaultBundlesV1.sol
```

```
29:     function vaultBundlesV1Deposit(  
    ..SNIP..  
48:         uint256 shares = IERC4626(vault).deposit(toDeposit, msg.sender);
```

```
File: VaultV2.sol
```

```
760:     function deposit(uint256 assets, address onBehalf) external returns  
    ↪ (uint256) {  
761:         accrueInterest();  
762:         uint256 shares = previewDeposit(assets);  
763:         enter(assets, shares, onBehalf);  
764:         return shares;  
765:     }  
    ..SNIP..  
776:     function enter(uint256 assets, uint256 shares, address onBehalf) internal {  
777:         require(canReceiveShares(onBehalf), ErrorsLib.CannotReceiveShares());  
778:         require(canSendAssets(msg.sender), ErrorsLib.CannotSendAssets());
```

For the send-assets gate, the address passed into the gate is always the `msg.sender`, which points to the `VaultBundlesV1` contract address. Assume that the `Vault V2`'s owner has already whitelisted the `VaultBundlesV1` contract address. In this case, the deposit via the bundle will go through.

However, it is not possible for the send-assets gate to filter the request via the user (token-sender address), since it only sees the bundle address. So, if Bob is not whitelisted to send assets but is whitelisted to receive shares, he can use the bundle contract to bypass the restriction.

Impact

Users prohibited from supplying assets can deposit or migrate through the whitelisted bundle, bypassing the vault's depositor access-control or compliance policy.

Code Snippet

<https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/blob/f4c5add4a7a71420f06696d030e31f8803f76776/VaultBundlesV1.sol#L16>

Tool Used

Manual Review

Recommendation

Consider implementing a `initiator()` returning the original caller.

Discussion

bhargavbh

fixed in <https://github.com/morpho-org/bundles/pull/90>

Issue L-2: Per-market rounding can revert force withdrawals with exact allowances [RE-SOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/issues/10>

Vulnerability Detail

The force-withdraw paths size the whole deallocation from one aggregate formula below, regardless of the number of loops or "per-market chunks"

```
File: VaultForceWithdrawBundlesV1.sol
114:         uint256 assetsToDeallocate = forceWithdrawAssets.mulDivDown(WAD, WAD +
    ↪     penalty);
```

So, there is a slight formula mismatch if there are more than 1 loop or market.

Assume the following:

- $F = \text{forceWithdrawAssets}$
- $p = \text{penalty}$
- $P = \text{assetsToDeallocate} = \text{floor}(F * WAD / (WAD + p))$
- $\text{assets}[i]$ the per-market chunks ($\sum \text{assets}[i] = P$)

What the bundle budgets:

```
charge = P + ceil(P * p / WAD)
```

What execution charges: one round-up per market (VaultV2.forceDeallocate ceils per call)

```
charge = P +  $\sum$  ceil(assets[i] * p / WAD)
```

For example, 2% penalty, $F = 100$ base units, adapter position spread over 4 markets (chunks 25/25/25/23, $P = 98$):

```
budgeted: 98 + ceil(98 * 0.02) = 98 + 2 = 100 (OK)
actual: 98 + ceil(0.50) + ceil(0.50) + ceil(0.50) + ceil(0.46)
      = 98 + 1 + 1 + 1 + 1 = 98 + 4 = 102 > F
```

So, there is a dust overcharge of ($\leq n - 1$ where n is the number of markets). The dust is economically insignificant.

However, if a user gives an allowance/permit sized for exactly F worth of shares, the transaction will revert due to cumulative rounding.

The same behavior also happens on the withdrawals themselves. Each `VaultV2.withdraw` rounds the shares to burn up per call ($\sum \text{ceil}(\text{shares}[i]) \quad \text{ceil}(\sum \text{shares}[i])$)

So, the caller should provision permits/allowances with a small buffer if they want to workaround this issue.

Impact

Users who are not aware of this behavior might grant an allowance/permit sized for exactly F worth of shares, which may cause the transaction to revert due to cumulative rounding.

Code Snippet

<https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/blob/dfed81d3773e1c7e9caaaf8cbb02f73abae46cc3/VaultForceWithdrawBundlesV1.sol#L114>

Tool Used

Manual Review

Recommendation

After further discussion, it was understood that the following comments attempt to cover the above scenario:

```
File: VaultForceWithdrawBundlesV1.sol
```

```
94:      /// @dev Requires the sender to have enough shares to withdraw ceil(assets  
↪ * penalty / WAD) and then assets, for each market in the list, where the sum of  
↪ the assets is equal to assetsToDeallocate.
```

```
File: VaultForceWithdrawBundlesV1.sol
```

```
43:      /// @dev The sender must have given enough allowance over vault shares to  
↪ this bundler, beforehand or via sharesPermit.
```

```
44:      /// @dev Requires Morpho Blue to have more than the assets in liquidity.
```

However, it is recommended to improve the NatSpec and explicitly document that per-market rounding may require a slightly larger share allowance, so callers and integrators can provide sufficient allowance.

Discussion

bhargavbh

addressed in <https://github.com/morpho-org/bundles/pull/96/changes>

Issue L-3: Redundant oracle read blocks full position closure when market's oracle is down [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/issues/11>

Vulnerability Detail

One of the test cases attempts to perform a full position migration when the source oracle is down.

```
File: BlueBundlesTest.sol
1054:    /// @dev The source oracle is never read during a full-position
    ↪ migrateBorrowPosition: the debt is zero by the
1055:    /// time the collateral withdrawal health check runs, which
    ↪ short-circuits before the oracle call. Positions can
1056:    /// therefore migrate out of a market whose oracle is broken.
1057:    function testMigrateBorrowPositionWithBrokenSourceOracle() public {
```

Let's assume that a market's oracle is down, and a user wants to be able to perform a full repayment (`borrowShares == 0`) and withdraw all collateral via `blueBundlesV1RepayAndWithdrawCollateral()`.

Currently, it will not work because it will revert when `.price()` is triggered. Even if the position's `borrowShares` is zero, it will still go through the following steps, which is unnecessary because it can be short-circuited, since the last requirement will always be true when `borrowShares == 0`.

```
File: BlueBundlesV1.sol
290:        uint256 borrowed =
    ↪ uint256(position.borrowShares).toAssetsUp(market.totalBorrowAssets,
    ↪ market.totalBorrowShares);
291:        uint256 price = IOracle(marketParams.oracle).price();
```

```
292:         uint256 maxBorrow = uint256(position.collateral).mulDivDown(price,  
↪  ORACLE_PRICE_SCALE).mulDivDown(maxLtv, WAD);  
293:         require(borrowed <= maxBorrow, LtvExceeded());
```

Impact

Full repayment and withdrawing all collateral will revert when market's oracle is down.

Code Snippet

<https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/blob/075883242b341b0d90dd0eddf8d7cab826644bb/BlueBundlesV1.sol#L290>

Tool Used

Manual Review

Recommendation

If the intention is to allow the user to perform a full repayment (`borrowShares == 0`) and withdraw all collateral via `blueBundlesV1RepayAndWithdrawCollateral()` when oracle is down, consider adding the following short-circuit, similar to how Blue's `_isHealthy` does [here](#)

```
function requireMaxLtv(MarketParams memory marketParams, address sender,  
↪  uint256 maxLtv) internal view {  
    if (maxLtv >= marketParams.lltv) return;  
    Market memory market = IMorpho(BLUE).market(marketParams.id());  
    Position memory position = IMorpho(BLUE).position(marketParams.id(),  
↪  sender);  
+    if (position.borrowShares == 0) return;  
    uint256 borrowed =  
↪  uint256(position.borrowShares).toAssetsUp(market.totalBorrowAssets,  
↪  market.totalBorrowShares);  
    uint256 price = IOracle(marketParams.oracle).price();
```

Discussion

bhargavbh

fixed in <https://github.com/morpho-org/bundles/pull/75>

Issue L-4: Pure borrowing via blueBundlesV1SupplyCollateralAndBorrow() is not supported [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/issues/12>

Vulnerability Detail

```
File: Morpho.sol
303:     function supplyCollateral(MarketParams memory marketParams, uint256
    ↪ assets, address onBehalf, bytes calldata data)
304:         external
305:     {
306:         Id id = marketParams.id();
307:         require(market[id].lastUpdate != 0, ErrorsLib.MARKET_NOT_CREATED);
308:         require(assets != 0, ErrorsLib.ZERO_ASSETS);
309:         require(onBehalf != address(0), ErrorsLib.ZERO_ADDRESS);
```

The blueBundlesV1SupplyCollateralAndBorrow always calls the supplyCollateral() function. Notice that the supplyCollateral will revert if assets == 0.

In this case, a user with existing collateral who wants a pure borrow (collateralAssets = 0, borrowAssets > 0) always reverts before the borrow leg is reached.

Impact

A user attempting to perform a pure borrowing (collateralAssets = 0, borrowAssets > 0) via the bundle's blueBundlesV1SupplyCollateralAndBorrow function will encounter a revert.

Code Snippet

<https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/blob/075883242b341b0d90dd0eddf8d7cab826644bb/BlueBundlesV1.sol#L66>

Tool Used

Manual Review

Recommendation

Consider wrapping the `TokenLib.forceApproveMax` and `IMorpho(BLUE).supplyCollateral` within `if (collateralAssets > 0) { ...code... }`.

Discussion

bhargavbh

fixed in <https://github.com/morpho-org/bundles/pull/57>

Issue L-5: `vaultForceWithdrawBundlesV1LiquidVaultV2()` can be not usable within Morpho Blue flash loan [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/issues/13>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`vaultForceWithdrawBundlesV1LiquidVaultV2()` sizes every Blue withdrawal from `totalSupplyAssets - totalBorrowAssets`. That figure overstates what Morpho Blue can actually pay whenever the Blue's loan token balance is transiently below its booked free liquidity, the base case being execution inside a Morpho Blue flash loan callback. The subsequent withdrawal tries to fetch from Blue more than it holds, so such a call reverts.

Vulnerability Detail

Inside a flash loan callback Blue's real balance is lower than `totalSupplyAssets - totalBorrowAssets` accounting implies. The deallocation loop sets `availableToWithdraw = min(adapterAssets, totalSupplyAssets - totalBorrowAssets)` (line 199) and deallocates this amount. The following `IVaultV2's withdraw()` (line 180) and `forceDeallocate()` (line 203) try to transfer the computed amount from Blue, reverting the call.

Impact

The force withdraw cannot be composed inside a Morpho Blue flash loan, or inside any transaction that lowers Blue's loan-token balance below a market's booked free liquidity. In those contexts it reverts instead of withdrawing, making the function unavailable where an integrator would chain it atomically. Funds are not lost and the revert is atomic, net effect being a composability and availability limitation.

Code Snippet

Liquidity adapter sizing is bounded by accounting free liquidity only:

VaultForceWithdrawBundlesV1.sol#L174-L177

```
(uint256 totalSupplyAssets, uint256 totalSupplyShares, uint256 totalBorrowAssets,) =  
    MorphoBalancesLib.expectedMarketBalances(IMorpho(BLUE), liquidityMarketParams);  
uint256 liquidityAdapterAssets =  
    ↪ liquidityAdapterShares.toAssetsDown(totalSupplyAssets, totalSupplyShares);  
withdrawableAssets += UtilsLib.min(liquidityAdapterAssets, totalSupplyAssets -  
    ↪ totalBorrowAssets);
```

Deallocation loop sizing use the same bound:

VaultForceWithdrawBundlesV1.sol#L196-L199

```
(uint256 totalSupplyAssets, uint256 totalSupplyShares, uint256 totalBorrowAssets,) =  
    MorphoBalancesLib.expectedMarketBalances(IMorpho(BLUE), marketParams);  
uint256 adapterAssets = adapterShares.toAssetsDown(totalSupplyAssets,  
    ↪ totalSupplyShares);  
uint256 availableToWithdraw = UtilsLib.min(adapterAssets, totalSupplyAssets -  
    ↪ totalBorrowAssets);
```

Tool Used

Manual Review

Recommendation

Consider either documenting the limitation or capping each market's contribution by the Morpho Blue's remaining balance, as MetaMorpho does, e.g.:

VaultForceWithdrawBundlesV1.sol#L177

```
-         withdrawableAssets += UtilsLib.min(liquidityAdapterAssets,  
    ↪ totalSupplyAssets - totalBorrowAssets);  
+         withdrawableAssets += UtilsLib.min(  
+         liquidityAdapterAssets,
```

```
+             UtilsLib.min(totalSupplyAssets - totalBorrowAssets,  
↪   IERC20(asset).balanceOf(BLUE))  
+         );
```

VaultForceWithdrawBundlesV1.sol#L199

```
-         uint256 availableToWithdraw = UtilsLib.min(adapterAssets,  
↪   totalSupplyAssets - totalBorrowAssets);  
+         uint256 availableToWithdraw = UtilsLib.min(  
+             adapterAssets,  
+             UtilsLib.min(totalSupplyAssets - totalBorrowAssets,  
↪   IERC20(asset).balanceOf(BLUE))  
+         );
```

Discussion

bhargavbh

acknowledged. we mention in the README that "bundles are user-facing and are not meant to be called by other contracts".

Issue L-6: VaultBundlesV1's and VaultForceWithdrawBundlesV1's balance of any token can be stolen via fake vault and adapter [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/issues/15>

Summary

VaultBundlesV1's `vaultBundlesV1Withdraw()`, VaultForceWithdrawBundlesV1's `vaultForceWithdrawBundlesV1IlliquidVaultV1()`, `vaultForceWithdrawBundlesV1IlliquidVaultV2()`, and `vaultForceWithdrawBundlesV1LiquidVaultV2()` accept an arbitrary vault (and adapter) and validate only spoofable getters. A contract returning attacker-chosen values can make every vault-side call a no-op, while the remaining real operation, an ERC-20 transfer or a Morpho Blue supply, move the bundler's own token balance to the attacker. As `asset = IERC4626(vault).asset()` is vault returned, any balance the bundler happens to hold can be drained by any caller.

Vulnerability Detail

The bundlers are designed to hold no funds between transactions, and their safety argument is that only `msg.sender`'s tokens are pulled and are fully spent in-transaction ([VaultBundlesV1.sol#L17](#)). That argument breaks once a balance is present, because the vault is caller-supplied and unauthenticated. `vaultBundlesV1Withdraw()` checks nothing about the vault; the force-withdraw paths check only getters a fake contract returns at will. `permitShares` is skipped on an empty signature, so the attacker needs no shares ([VaultBundlesV1.sol#L139-L142](#)).

`vaultBundlesV1Withdraw()`: the fake vault's `redeem` returns an attacker-chosen assets and moves nothing, `asset()` returns the real token the bundler holds, and the two transfers then pay that token out of the bundler to `referralFeeRecipient` and

`msg.sender`. With `shares = 1`, `minSharePriceE27 = 0`, `referralFeePct = 0`, the attacker sets assets to the bundler's balance and takes it.

`vaultForceWithdrawBundlesV1LiquidVaultV2()`: with a fake vault reporting zero idle balance and no liquidity adapter, and a fake adapter, `assetsToWithdraw` becomes zero and `assetsToDeallocate` becomes `forceWithdrawAssets`; the deallocation loop burns that to zero using a real high-liquidity market's booked liquidity as the numeric cap while `forceDeallocate` on the fake vault is a no-op; `asset()` returns the target token; the final transfers pay `forceWithdrawAssets` of the bundler's balance to `msg.sender`.

`vaultForceWithdrawBundlesV1IlliquidVaultV2()`: the Blue supply of `forceWithdrawAssets` is real and pulls the loan token from the bundler crediting supply shares to `msg.sender`; the `onMorphoSupply` callback's `forceDeallocate` and `withdraw` are no-ops on the fake vault, so the bundler funds the supply from its own balance. The attacker points the market at any real Blue market for the held token.

`vaultForceWithdrawBundlesV1IlliquidVaultV1()`: the flash loan is balanced, and the callback's Blue supply moves the bundler's balance to `msg.sender`, capped by the fake vault's real Blue supply position read at `expectedSupplyAssets`. This variant therefore additionally requires the attacker to seed that position beforehand, which is recoverable after the attack.

A residual balance can arise at least in two ways: a direct or donation transfer, and a referral fee whose recipient is the bundler's own address, which stays in the contract on each fee-paying deposit, withdraw, or migrate.

Impact

Any token balance held by either bundler is stolen by anyone in a single permissionless call, unconditionally for `vaultBundlesV1Withdraw()` and the liquid Vault V2 path, and for any token with a real Blue market for the illiquid Vault V2 path. No funds are at risk in normal operation because the contracts hold nothing between transactions, so the loss is bounded by whatever residual is present.

Code Snippet

`vaultBundlesV1Withdraw`, fake vault redeem returns the amount and the transfers pay it from the bundler: [VaultBundlesV1.sol#L77-L84](#)

```

if (assets > 0) shares = IERC4626(vault).withdraw(assets, address(this),
    ↪ msg.sender);
else assets = IERC4626(vault).redeem(shares, address(this), msg.sender);
require(assets.mulDivDown(1e27, shares) >= minSharePriceE27, SlippageExceeded());

address asset = IERC4626(vault).asset();
uint256 referralFeeAssets = assets.mulDivDown(referralFeePct, WAD);
if (referralFeeAssets > 0) SafeTransferLib.safeTransfer(asset,
    ↪ referralFeeRecipient, referralFeeAssets);
SafeTransferLib.safeTransfer(asset, msg.sender, assets - referralFeeAssets);

```

Liquid Vault V2, mutable guards a transfer of the bundler's asset to `msg.sender`: [VaultForceWithdrawBundlesV1.sol#L160-L161](#) [VaultForceWithdrawBundlesV1.sol#L208-L211](#)

```

uint256 withdrawn = assetsToWithdraw + assetsToDeallocate;
uint256 referralFeeAssets = withdrawn.mulDivDown(referralFeePct, WAD);
if (referralFeeAssets > 0) SafeTransferLib.safeTransfer(asset,
    ↪ referralFeeRecipient, referralFeeAssets);
SafeTransferLib.safeTransfer(asset, msg.sender, withdrawn - referralFeeAssets);

```

Illiquid Vault V2, real Blue supply funded by the bundler while the callback is a no-op: [VaultForceWithdrawBundlesV1.sol#L124-L127](#)

```

if (assets > 0) {
    bytes memory data = abi.encode(vault, adapter, marketParamsList[i], msg.sender);
    IMorpho(BLUE).supply(marketParamsList[i], assets, 0, msg.sender, data);
}

```

Tool Used

Manual Review

Recommendation

Consider rejecting a referral recipient equal to the bundler contract on every fee-paying entry point, removing the controllable accumulation route, e.g. declare `InvalidReferral`

FeeRecipient() and add the guard beside the existing referralFeePct check:

VaultBundlesV1.sol#L73

```
require(referralFeePct < WAD, PctExceeded());  
+ require(referralFeeRecipient != address(this),  
↪ InvalidReferralFeeRecipient());
```

Apply the same line at vaultBundlesV1Deposit ([VaultBundlesV1.sol#L39](#)), vaultBundlesV1 Migrate ([VaultBundlesV1.sol#L109](#)), and the liquid Vault V2 path ([VaultForceWithdrawBundlesV1.sol#L162](#)).

This guard does not stop the donation route, which remains sweepable. The complete fix is to authenticate the vault and adapter against the Morpho vault factory and registry.

Discussion

QGarchery

Yes this is known, and documented in the comment "This contract can approve tokens to arbitrary addresses. This is safe because a token amount pulled is always fully spent in the same transaction, and because the only tokens pulled to this contract are owned by msg.sender"

Still, we made it more explicit in <https://github.com/morpho-org/bundles/pull/97>. So we can consider this one as fixed

Issue L-7: BlueBundlesV1::blueBundlesV1RepayAndWithdrawCollateral() may transfer a null leftover [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-blue-bundles-july-13th-2026/issues/16>

Summary

The leftover transfer in `BlueBundlesV1::blueBundlesV1RepayAndWithdrawCollateral()` is not skipped when null, unlike the referral fee transfer above it (and on the rest of the codebase).

Vulnerability Detail

`maxRepayAssets - repayAssets - referralFeeAssets` is 0 whenever `maxRepayAssets` exactly covers the repaid assets plus the fee, which is the natural sizing on an exact assets repay with a null referral fee (`maxRepayAssets == repayAssets`).

Impact

Tokens reverting on 0 transfer are not supported, so the only impact is consistency throughout the codebase.

Code Snippet

[BlueBundlesV1.sol#L118-L120](#)

```
SafeTransferLib.safeTransfer(  
    marketParams.loanToken, msg.sender, maxRepayAssets - repayAssets -  
    ↪ referralFeeAssets  
);
```

Tool Used

Manual Review

Recommendation

Skip the transfer when the amount is null, consistent with the rest of the codebase.

Discussion

bhargavbh

acknowledged/no-fix. we consider it covered by the contract-level natspec in BlueBundlesV1.sol: `/// @dev Zero checks are not systematically performed.`

Disclaimers

Blackthorn does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.